

SANRAKSHAK

AI-Powered Website Security Assessment Platform

"Protect. Analyze. Secure."

A full-stack web application that scans any website for security vulnerabilities using industry-standard tools (Nmap, Nikto, OWASP ZAP), then uses AI to explain findings in simple language — all through a clean, modern dashboard.

Built by Vrinda | v1.0 | 2026

Table of Contents

1. What is SANRAKSHAK? Page 3

2. Why I Built It Page 3

3. How It Works — Full Flow Page 4

4. Tech Stack (Simple Breakdown) Page 6

5. Architecture Overview Page 7

6. Key Features Page 8

7. What Makes It Unique Page 10

8. Security Score System Page 11

9. Database Design Page 12

10. Interview Q&A Page 13

1 What is SANRAKSHAK?

SANRAKSHAK (Hindi for "Protector" or "Guardian") is an **AI-powered website security scanner**. You type in any website URL, and it automatically checks that website for security problems — open ports, server weaknesses, web vulnerabilities like SQL injection and XSS — and then gives you a clean report with a security score out of 100.

> **In simple words:** Think of it like a health checkup for websites. Just like a doctor runs tests on your body, SANRAKSHAK runs security tests on websites and tells you what's wrong — and how to fix it.

2 Why I Built It

The Problem

Website security tools like `Nmap`, `Nikto`, and `OWASP ZAP` are incredibly powerful — but they're **command-line tools**. They spit out walls of technical text. Only cybersecurity experts can read and understand them.

Meanwhile, millions of website owners — students, freelancers, small businesses, startups — deploy websites without any clue about their security. They don't know that a single open port or missing security header could lead to a data breach.

The Solution

SANRAKSHAK bridges this gap. It wraps these professional CLI tools behind a **simple web interface** and uses **Google Gemini AI** to translate technical findings into plain English that anyone can understand.

Target Users

Students, freelancers, startup founders, small business owners who want to know if their website is secure.

Core Idea

Make security scanning as easy as typing a URL and clicking a button — no terminal required.

Value Add

AI translates jargon into plain English. A non-technical person can read and act on the report.

3 How It Works — Full Flow

Here's the complete flow from the moment a user types a URL to getting a full security report:



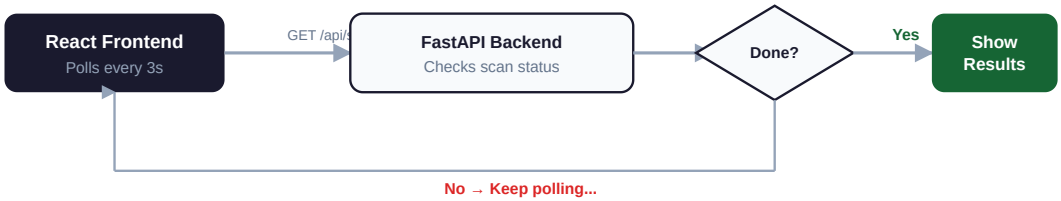
Step-by-Step Explanation

1. **User enters a URL** on the frontend (e.g., `example.com`). The frontend auto-adds `https://` if missing.
2. **Auth check** — The frontend checks if the user is logged in. If not, redirect to login.
3. **FastAPI receives the request** at `POST /api/scan`. It validates the URL.
4. **A scan record is created** in the database with status `"running"`.
5. **The scan ID is returned immediately** — the user doesn't have to wait. This is key to the UX.
6. **A background task starts** using Python's `asyncio.create_task()`. This runs independently of the API response.
7. **Three scanners run in parallel** (Nmap, Nikto, OWASP ZAP) using `asyncio.gather()`. Each runs in a separate thread via `run_in_executor` because they are CLI tools.
8. **Results are parsed** — raw output is converted into structured vulnerabilities with severity levels.
9. **Security score is calculated** based on weighted penalties for each severity level.
10. **Gemini AI generates a summary** — a human-readable explanation of the findings.

11. **Results are saved to the database** with status `"completed"`.

How the Frontend Gets Results (Polling)

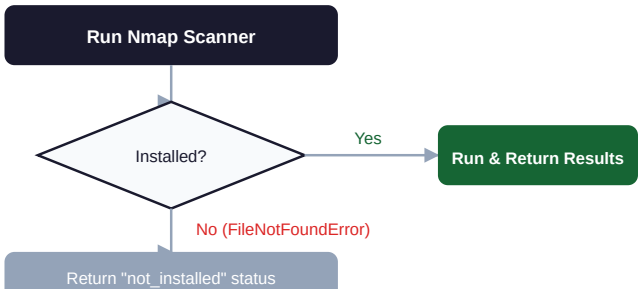
Since the scan runs in the background, the frontend **polls** (checks) the server every 3 seconds:



> **Why polling instead of WebSockets?** Polling is simpler to implement and debug. For a scanning tool where results update every few seconds (not in real-time like chat), polling every 3 seconds is perfectly fine and much easier to scale.

Graceful Tool Degradation

What happens if one of the scanning tools is not installed? The system **doesn't crash**. Each tool is wrapped in a try/except block:



This same pattern applies to Nikto and OWASP ZAP. The scan completes with partial results instead of failing entirely.

4 Tech Stack — Simple Breakdown

🎨 Frontend (What Users See)

Tool	What It Does
React 19	Builds the user interface — buttons, forms, pages
TypeScript	JavaScript with types — catches errors before they happen
Vite 8	Super-fast build tool — compiles code in milliseconds
Tailwind CSS 4	Style the app using utility classes — no custom CSS files
Shadcn UI	Pre-built, beautiful components (buttons, cards, inputs)
Axios	Sends HTTP requests to the backend API
jsPDF	Generates PDF reports right in the browser

⚙️ Backend (Server-Side Logic)

Tool	What It Does
FastAPI	Python web framework — handles API routes
SQLAlchemy 2.0	ORM — talks to the database using Python objects
Alembic	Database migration tool — manages schema changes
Pydantic	Data validation — ensures requests have correct shapes
bcrypt	Password hashing — never stores plain-text passwords
python-jose	JWT tokens — keeps users logged in securely
httpx	Async HTTP client — makes external API calls

🔍 Security Scanning Tools

Tool	What It Scans For
Nmap	Open ports, running services, OS detection
Nikto	Web server misconfigs, dangerous files, outdated software

💻 Infrastructure

Tool	What It Does
PostgreSQL 16	Relational database — stores users, scans, vulnerabilities
Docker	Containerizes the backend with all tools pre-installed
Docker Compose	Runs backend + database together with one command

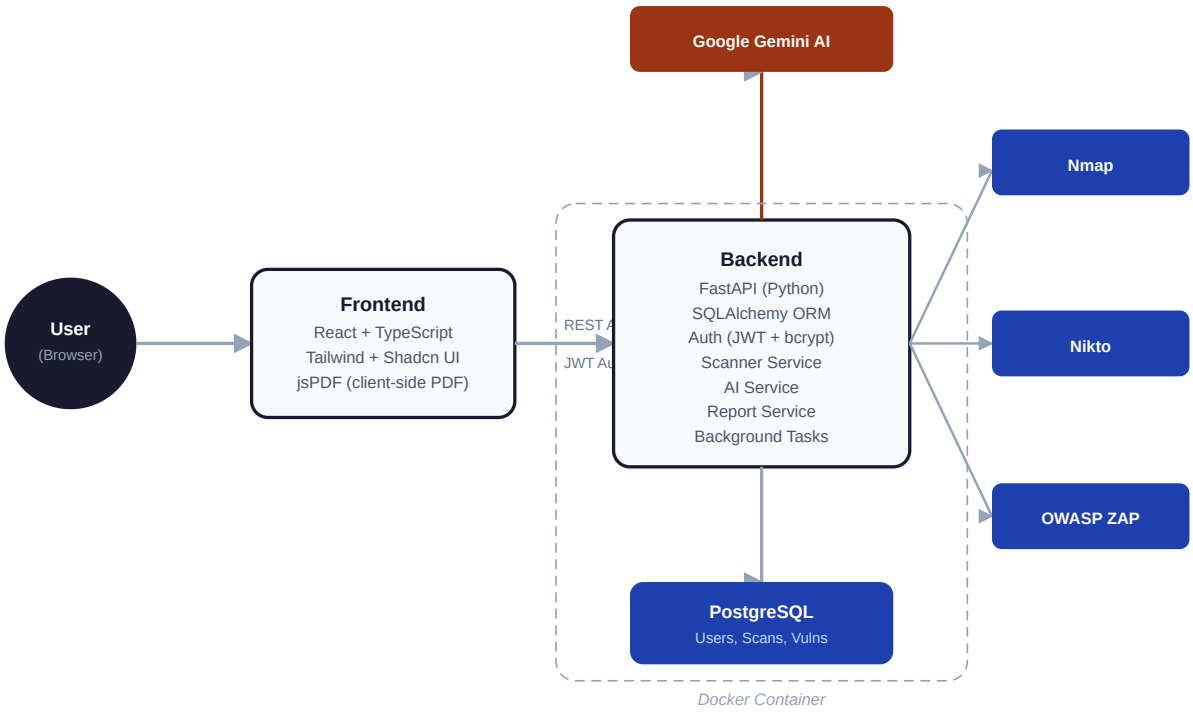
OWASP ZAP	SQL injection, XSS, security header issues

Google Gemini	AI model that generates human-readable security summaries

💡 **Why FastAPI over Django/Flask?** FastAPI is async-native, meaning it can handle multiple requests concurrently without blocking. Since SANRAKSHAK runs long-running scans (up to 10 minutes), async support is critical. It's also 2-3x faster than Flask for I/O-bound tasks.

5 Architecture Overview

Here's how all the pieces connect together:



Key architectural decisions:

- **Frontend and Backend are separate** — can be deployed independently, developed by different teams
- **Scanners run as subprocesses** — not libraries. The backend spawns OS-level processes.
- **AI is a separate service call** — easily swappable (could replace Gemini with OpenAI)
- **Everything runs in Docker** — one command to start the entire system

6 Key Features

1. User Authentication

Registration & Login

Email + password registration. Passwords are hashed with **bcrypt** before storing. JWT token issued on login (24-hour expiry). Frontend stores token in `localStorage`.

Route Protection

Protected routes on frontend — unauthenticated users are redirected to `/login`. Axios interceptor auto-attaches JWT to every request. 401 responses trigger auto-logout.

2. Website Scanning

Enter any URL and get a comprehensive security scan. Three industry-standard tools run **concurrently**:

Nmap

Port scanning + service detection. Flags: `-Pn -sV -sC -T4`. Finds open ports and what software is running on them.

Nikto

Web server vulnerability scanner. Checks for outdated software, dangerous files, misconfigurations, missing headers.

OWASP ZAP

Full web app security testing. SQL injection, XSS, insecure headers, cookie issues. The industry standard for web app testing.

3. AI-Powered Reports (Premium Feature)

Raw security output is sent to **Google Gemini 2.0 Flash**, which generates a clean, plain-English summary of the findings. The AI is prompted to act as a "cybersecurity report writer" with temperature set to 0.3 for deterministic output. If no API key is configured or the call fails, a **template-based fallback** generates a reasonable summary automatically.

4. Dashboard

Overview

Security Score (animated SVG ring), total vulnerabilities found, total scans performed. All at a glance.

Scan History

Table of all past scans with color-coded status badges (running/completed/failed) and score indicators (green/yellow/red).

5. Scan Results (4 Tabs)

Tab	Content	Purpose

Raw Report	Terminal-style blocks with raw Nmap/Nikto/ZAP output	For experts who want the raw data
Text Report	Structured interpretation of each tool's findings	Middle ground — organized but detailed
Vulnerabilities	Full list grouped by severity + bar chart	Quick overview of what's critical
AI Report	Gemini-generated plain-English summary	For non-technical users (premium)

6. Report Export

Reports can be exported in three formats:

- **PDF** — Generated client-side using jsPDF (no server round-trip needed)
- **HTML** — Server-side generation with styled CSS
- **Markdown** — Server-side generation for sharing on GitHub/docs

7 What Makes SANRAKSHAK Unique

1. Non-Blocking Architecture

Most security scanners make you wait. SANRAKSHAK returns a scan ID **immediately** and runs everything in the background. The user can close the tab and come back later. This is possible because of Python's `asyncio.create_task()`.

2. Graceful Degradation

If Nmap, Nikto, or ZAP isn't installed, the system doesn't crash. It marks that scanner as "not installed" and continues with whatever tools are available. This makes it work in any environment.

3. AI + Fallback Hybrid

The AI summary has a template-based fallback. The app works **even without a Gemini API key**. This means you can develop and demo it for free, then add AI when you're ready for production.

4. Client-Side PDF Generation

Instead of a heavy server-side PDF library, jsPDF generates reports in the browser. This means faster downloads, less server load, and no need for tools like wkhtmltopdf in the Docker container.

5. Monochrome Design System

The entire UI uses a greyscale palette with `oklch` color space — zero chroma. This gives it a distinctive "security terminal" aesthetic that matches the product's identity.

6. Smart Scoring Algorithm

The 0-100 score uses weighted penalties: Critical (-25), High (-15), Medium (-8), Low (-3), Info (0). This heavily penalizes the most dangerous issues, guiding users to fix the worst problems first.

TL;DR for interviews: SANRAKSHAK is not just "a scanner with a UI." It's an **async background processing system** with **graceful degradation**, **AI-powered interpretation**, and a **modern full-stack architecture** that works even without all dependencies installed.

8 Security Score System

The security score is a number from **0 to 100** where 100 means "no vulnerabilities found" and 0 means "critical issues everywhere."

How the Score is Calculated

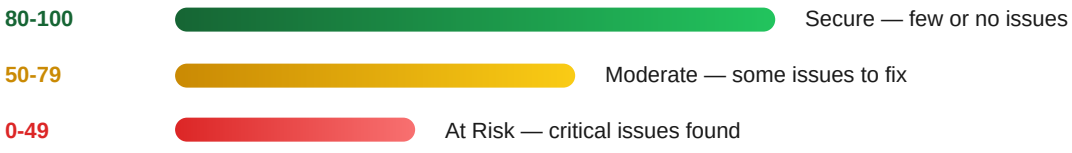
Severity	Points Deducted	Example
Critical	**-25** points each	Remote code execution, SQL injection
High	**-15** points each	Open admin ports, XSS vulnerabilities
Medium	**-8** points each	Missing security headers, outdated software
Low	**-3** points each	Minor information disclosure
Info	**0** points	Informational findings (not a risk)

Visual Example

If a scan finds 1 Critical, 2 Medium, and 3 Info findings:

```
Starting score: 100
- 1 Critical: -25 → 75
- 2 Medium: -16 → 59
- 3 Info: 0 → 59
Final Score: 59/100 (Medium Risk)
```

Score Color Mapping

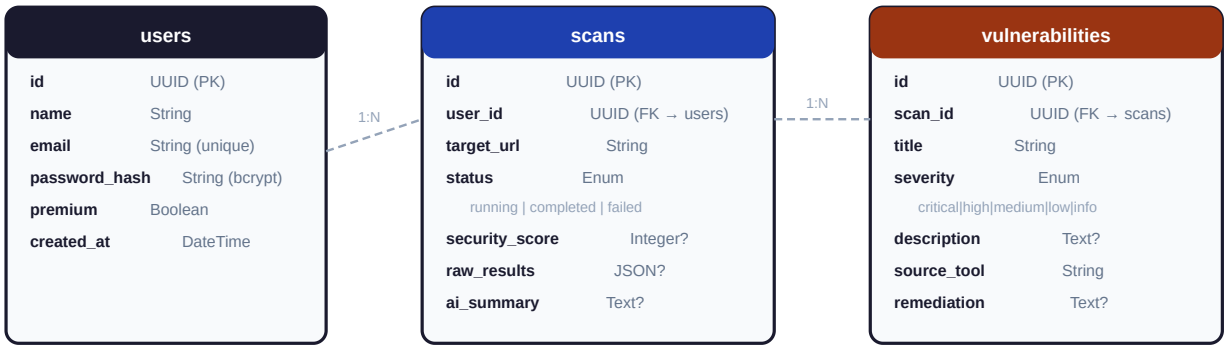


Deduplication

If multiple tools find the same vulnerability (e.g., both Nikto and ZAP detect the same missing header), it is counted only once. This prevents score inflation from duplicate findings.

9 Database Design

The PostgreSQL database has **three main tables**:



Relationships

- **One user** can have **many scans** (1:N relationship via `user_id` foreign key)
- **One scan** can have **many vulnerabilities** (1:N relationship via `scan_id` foreign key)
- **Vulnerabilities** store which tool found them (`source_tool`: nmap/nikto/zap)

10 Interview Q&A

Here are the most likely questions an interviewer will ask, with strong answers:

Q1: What is SANRAKSHAK and what problem does it solve?

SANRAKSHAK is an AI-powered website security assessment platform. The problem it solves is that security scanning tools like Nmap, Nikto, and OWASP ZAP are command-line tools that only experts can use. SANRAKSHAK wraps them behind a simple web UI and uses AI to explain findings in plain English, making security assessment accessible to non-technical users.

Q2: Why did you choose FastAPI over Django or Flask?

FastAPI is async-native by default, which is critical for this project because security scans take 1-10 minutes. With Flask (synchronous), a scan would block the entire thread. FastAPI handles concurrent requests while scans run in the background. It's also 2-3x faster for I/O-bound tasks, has automatic API documentation (Swagger UI), and uses Pydantic for built-in data validation.

Q3: How do you handle long-running scans without blocking the API?

When a scan is requested, I create the scan record in the database and return the scan ID immediately. Then I launch the actual scanning as a background task using Python's asyncio.create_task(). The frontend polls GET /api/scan/{id} every 3 seconds to check if the scan is complete. This keeps the API responsive and the user informed.

Q4: What happens if one of the scanning tools is not installed?

Each scanner is wrapped in a try/except block that catches FileNotFoundError. If a tool isn't installed, the system returns a "not_installed" status for that tool and continues with whatever tools are available. The scan completes with partial results instead of crashing. This graceful degradation makes the app work in any environment.

Q5: How does the AI summary work? What if there's no API key?

Vulnerabilities are formatted into a prompt and sent to Google Gemini 2.0 Flash with temperature 0.3 for deterministic output. If no API key is configured or the API call fails, a template-based fallback function generates a reasonable summary. This means the app works even without AI — the summary is less insightful but still functional.

Q6: How do you ensure the same vulnerability isn't counted twice?

After all three scanners finish, I deduplicate findings by comparing vulnerability titles. If Nmap and Nikto both detect the same issue, it's stored once with the source_tool field noting which tools found it. This prevents score inflation from duplicate findings.

Q7: Why use jsPDF for PDF generation instead of a server-side library?

Client-side PDF generation with jsPDF reduces server complexity — no need for wkhtmltopdf or weasyprint in the Docker container. It's faster for the user (no network round-trip), reduces server load, and the PDF is generated instantly in the browser. The server only needs to serve the data.

Q8: How does authentication work in your app?

On registration, passwords are hashed with bcrypt and stored in PostgreSQL. On login, the server verifies the password and returns a JWT token (24-hour expiry). The frontend stores the token in localStorage and an Axios interceptor attaches it to every request. Protected routes check for the token and redirect to login if missing. 401 responses trigger auto-logout.

Q9: How do you calculate the security score?

The score starts at 100 and deducts points based on severity: Critical (-25), High (-15), Medium (-8), Low (-3), Info (0). The score is floored at 0 and ceiling at 100. This heavily penalizes critical issues, guiding users to fix the most dangerous problems first. Duplicate findings are excluded from the calculation.

Q10: What would you improve or add next?

Three improvements: (1) Add WebSockets for real-time scan progress instead of polling. (2) Implement a refresh token mechanism — currently tokens expire in 24 hours with no refresh. (3) Add scheduled/recurring scans so users can monitor their website's security over time. Also, the Alembic migrations directory is empty — in production, I'd generate proper migration files instead of using create_all().

Q11: Why did you use Docker for this project?

Security tools like Nmap, Nikto, and ZAP need to be installed on the system. Docker ensures the backend container has all tools pre-installed, so anyone can run the project with just "docker compose up" without worrying about installing dependencies. The Dockerfile installs nmap and nikto directly in the Python image. Docker Compose orchestrates the API and PostgreSQL services with health checks.

Q12: Explain the frontend-backend communication flow.

The React frontend uses Axios with a base URL pointing to the FastAPI backend. An interceptor reads the JWT from localStorage and attaches it as a Bearer token. When a scan is requested, POST /api/scan sends the URL.

The backend returns a scan ID. The frontend then polls GET /api/scan/{id} every 3 seconds until status changes from "running" to "completed", then displays the results.

Pro tip for the interview: Always mention **why** you made a decision, not just **what** you did. For example: "I chose asyncio.create_task because scans take minutes and I can't block the API thread" is much stronger than "I used asyncio for background tasks."